

■ Event Inquiry Management API

Technical Round Interview Q&A; & Architecture Preparation Guide

1. Technology Stack & Elevator Pitch

Q1: Can you provide a high-level overview of the Event Inquiry Management API?

Answer: The Event Inquiry Management API is a stateless Spring Boot 3 RESTful microservice built using Java 17, Spring Security 6, Spring Data JPA, and PostgreSQL. It provides customer inquiry submission workflows for events (weddings, corporate summits, birthdays), role-based access control (USER vs ADMIN), and strict server-side IDOR (Insecure Direct Object Reference) authorization checks to prevent unauthorized cross-user data access. The project is fully containerized using Docker Compose and pre-configured for cloud deployment on Railway.

Component	Technology Chosen	Architectural Justification
Language & Runtime	Java 17 (LTS)	Long-Term Support, enhanced performance, pattern matching, record types.
Framework	Spring Boot 3.2.5	Enterprise standard, autoconfiguration, embedded Tomcat, Jakarta EE 10.
Security	Spring Security 6 + JWT	Stateless JWT authentication, BCrypt password hashing, RBAC annotations.
Database & ORM	PostgreSQL 16 / Hibernate	ACID compliance, JSONB support, relational indexing, JPA repositories.
API Specs	OpenAPI 3.0 / Swagger UI	Interactive API testing, Bearer JWT authentication scheme.

2. Database Schema & Entity Design Q&A;

Q2: Walk me through your database schema design. What entities exist and how are they related?

Answer: The relational database consists of two primary tables: **users** and **event_inquiries**.

- **users:** Stores account credentials (id, full_name, email [UNIQUE], password [BCrypt hash], role [ROLE_USER/ROLE_ADMIN], created_at).
- **event_inquiries:** Stores inquiry records (id, user_id [FK], customer_name, customer_email, customer_phone, event_type, event_date, location, estimated_budget, guest_count, status, special_requests).
- **Relationship:** 1-to-Many relationship (User 1 ■■> N EventInquiries) mapped via @ManyToOne(fetch = FetchType.LAZY) with foreign key user_id.

Q3: Why did you choose BigDecimal for the estimated_budget field instead of double or float?

Answer: Binary floating-point types (double and float) suffer from IEEE 754 precision rounding errors during arithmetic operations. In financial and budget domain modeling, exact precision is mandatory. BigDecimal maps cleanly to PostgreSQL NUMERIC(12,2), guaranteeing exact 2-decimal point precision without cumulative rounding discrepancies.

Q4: What indexes did you create and why?

Answer:

1. **Unique Index on users.email:** Enables O(1) time complexity lookup during user authentication and registration collision checks.
2. **Foreign Key Index idx_inquiries_user_id:** Speeds up user-owned inquiry listing queries (SELECT * FROM event_inquiries WHERE user_id = ?).
3. **Filtering Indexes idx_inquiries_status and idx_inquiries_event_type:** Optimizes administrative filtering and reporting queries.

3. Authentication & Security Architecture Q&A;

Q5: Explain your JWT authentication flow step-by-step.

Answer:

1. **Login Request:** User submits POST /api/v1/auth/login with email & password.
2. **Authentication Manager:** Spring Security's AuthenticationManager verifies password hash against stored BCrypt hash.
3. **Token Generation:** JwtTokenProvider generates an HMAC SHA-256 signed JWT string containing subject (email), issue date, 24-hour expiration, and granted authorities (ROLE_USER / ROLE_ADMIN).
4. **Request Interception:** JwtAuthenticationFilter intercepts subsequent HTTP requests, parses the Authorization: Bearer <token> header, validates signature, loads UserDetails, and populates SecurityContextHolder.

4. Insecure Direct Object Reference (IDOR) Prevention

Q6: How did you ensure that User A cannot access or edit User B's protected inquiry simply by changing an ID in the URL?

Answer: Authorization is enforced directly inside the service layer (EventInquiryServiceImpl) via an explicit resource ownership validation method (verifyOwnershipOrAdmin). When any resource method (getInquiryById, updateInquiry, deleteInquiry) is invoked, the service compares the owner ID of the entity against the authenticated user's ID:

```
private void verifyOwnershipOrAdmin(EventInquiry inquiry, User currentUser) {
    boolean isAdmin = currentUser.getRole() == Role.ROLE_ADMIN;
    boolean isOwner = Objects.equals(inquiry.getUser().getId(), currentUser.getId());

    if (!isAdmin && !isOwner) {
        throw new UnauthorizedAccessException("Access denied: You do not have permission to access or modify this inquiry.");
    }
}
```

If a non-admin user attempts an IDOR attack by altering the URL path parameter, the method throws UnauthorizedAccessException, which GlobalExceptionHandler catches and returns as **HTTP 403 Forbidden**. This was verified with automated integration tests in SecurityAuthorizationTest.java.

5. API Architecture & Standardized Error Handling

Q7: How is global error handling implemented across the REST APIs?

Answer: We use @RestControllerAdvice in GlobalExceptionHandler.java to capture all exceptions framework-wide. Validation errors (MethodArgumentNotValidException) extract field errors into a key-value map, while domain exceptions map to appropriate HTTP status codes (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict).

6. Testing & Deployment Strategy

Q8: How did you structure your test suite?

Answer: The project includes 9 comprehensive JUnit 5 tests:

- **`SecurityAuthorizationTest`**: Integration test using MockMvc and @WithMockUser to verify IDOR protection and role enforcement.
- **`AuthControllerTest`**: MockMvc tests verifying registration and JWT login response contracts.
- **`EventInquiryServiceTest`**: Unit tests covering service business logic and repository interactions.

Tests run against an isolated H2 database profile (`test`), completing build execution with zero failures.

Q9: How is the application deployed?

Answer:

1. **Local Dev Mode**: Uses embedded H2 database (dev profile) running on `http://localhost:8080`.
2. **Docker Compose**: Multi-container setup with PostgreSQL 16 container (``docker-compose up --build``).
3. **Railway Cloud**: Pre-configured `railway.json` and multi-stage Dockerfile with dynamic `$PORT` binding.